Facultad de Ingeniería
Departamento de Ciencias de la Computación
CC3069 — Computación Paralela y Distribuida

Proyecto 2
Fuerza bruta DES con MPI

Informe de implementación y análisis de rendimiento

Jorge López	221038
Giovanni Santos	22523
Angel Herrarte	22873

Guatemala, 14 de octubre de 2026

Índice

Contenido	Página
1. Introducción	2
2. Antecedentes	3
2.1. Diagrama de DES	4
3. Implementación y rutinas	5
3.1. Diagramas de rutinas	6
4. Flujo del programa MPI	7
4.1. Búsqueda y terminación	8
4.2. Primitivas y reparto	9
5. Método experimental	10
6. Resultados con llaves controladas	11
7. Descifrado de otros equipos	12
8. Profiling	13
9. Retos y verificación	14
10. Conclusiones y recomendaciones	14
Referencias	15
Anexo 1. Catálogo de funciones	16
Anexo 1. Catálogo de funciones (continuación 1)	17
Anexo 1. Catálogo de funciones (continuación 2)	18
Anexo 2. Bitácora: casos controlados	19
Anexo 2. Bitácora: continuación	20
Anexo 2. Bitácora: textos externos	21
Apéndice. Evidencia y reproducción	22

1. Introducción

Implementamos una búsqueda de llaves DES mediante fuerza bruta en C++17 con una versión secuencial y otra paralela con OpenMPI. El criterio de éxito es la presencia de una palabra o frase conocida en el descifrado. Comparamos los repartos por bloques y cíclico, medimos el efecto de la posición de la llave y realizamos profiling.

El trabajo experimental comprende 270 mediciones con llaves controladas, 180 mediciones de dos textos externos y 72 perfiles MPI instrumentados. Las búsquedas recuperaron los textos de los grupos 1 y 7 dentro de sus rangos publicados. Los resultados se respaldan con CSV, logs, hashes de archivos y versiones del entorno.

Objetivos

- Recuperar una llave candidata que produzca la frase conocida.
- Coordinar búsqueda y terminación mediante mensajes MPI.
- Comparar speedup, eficiencia y trabajo realizado entre repartos.
- Identificar las rutinas y fases que concentran el tiempo.

Alcance

Las mediciones se ejecutaron en una sola máquina. Los casos controlados usan un rango reducido de 2^22; los textos externos conservan los rangos publicados. No se recorrió exhaustivamente el espacio de 2^56 ni se evaluó un clúster de varias máquinas.

2. Antecedentes

DES es un cifrador simétrico por bloques: cifra y descifra con la misma llave. Trabaja con bloques de 64 bits y llaves de 56 bits efectivos; los ocho bits adicionales de su representación se usan para paridad. El algoritmo aplica 16 rondas [1, 2]. La publicación FIPS 46-3 fue retirada en 2005; aquí se estudia DES como problema académico de búsqueda [1].

Cifrado y descifrado

Después de la permutación inicial, el bloque se divide en mitades de 32 bits. Cada ronda combina una mitad con una subllave: $L_i = R_{i-1}$ y $R_i = L_{i-1}$ XOR $f(R_{i-1}, K_i)$. La función f expande, combina con XOR, sustituye mediante cajas S y permuta. La salida usa intercambio de mitades y permutación inversa. El descifrado aplica las subllaves en orden inverso [1].

El programa utiliza DES-ECB: cada bloque se transforma independientemente [3, 4]. Para cifrar un archivo se valida su longitud, se añade relleno hasta completar bloques de ocho bytes, se prepara la llave y se transforma cada bloque. Para descifrar se usa el mismo calendario con el modo inverso. El cifrador del proyecto añade espacios; las búsquedas conservan el relleno recibido.

Fuerza bruta y espacio numérico

El espacio completo contiene 2^56 = 72 057 594 037 927 936 llaves efectivas. La fuerza bruta prueba candidatos hasta detectar la frase o agotar el rango. En una búsqueda incremental desde cero, una solución única con índice k requiere k + 1 intentos. A una tasa hipotética de un millón de llaves por segundo, recorrer el espacio completo tomaría aproximadamente 2 283 años; esta cifra es una estimación aritmética, no una medición del equipo.

El costo de un candidato crece con el número de bloques del texto, pues esta implementación descifra todo el archivo antes de buscar la frase. Para rangos reducidos y un texto fijo, comparar tiempos exige mantener archivo, frase, límite y compilación. Una frase distintiva reduce coincidencias accidentales, pero el resultado sigue siendo una llave candidata reconocida por substring.

Paralelismo y métricas

MPI distribuye candidatos entre procesos con memoria separada. El speedup se calcula como $S = \text{promedio}(T_{seq}) / \text{promedio}(T_{par})$; la eficiencia es $E = S/N$. La terminación temprana puede hacer que los repartos prueben cantidades distintas de llaves. Por eso un speedup superior a N debe analizarse junto a los intentos, y no interpretarse directamente como utilización de recursos.

2.1. Diagrama de DES

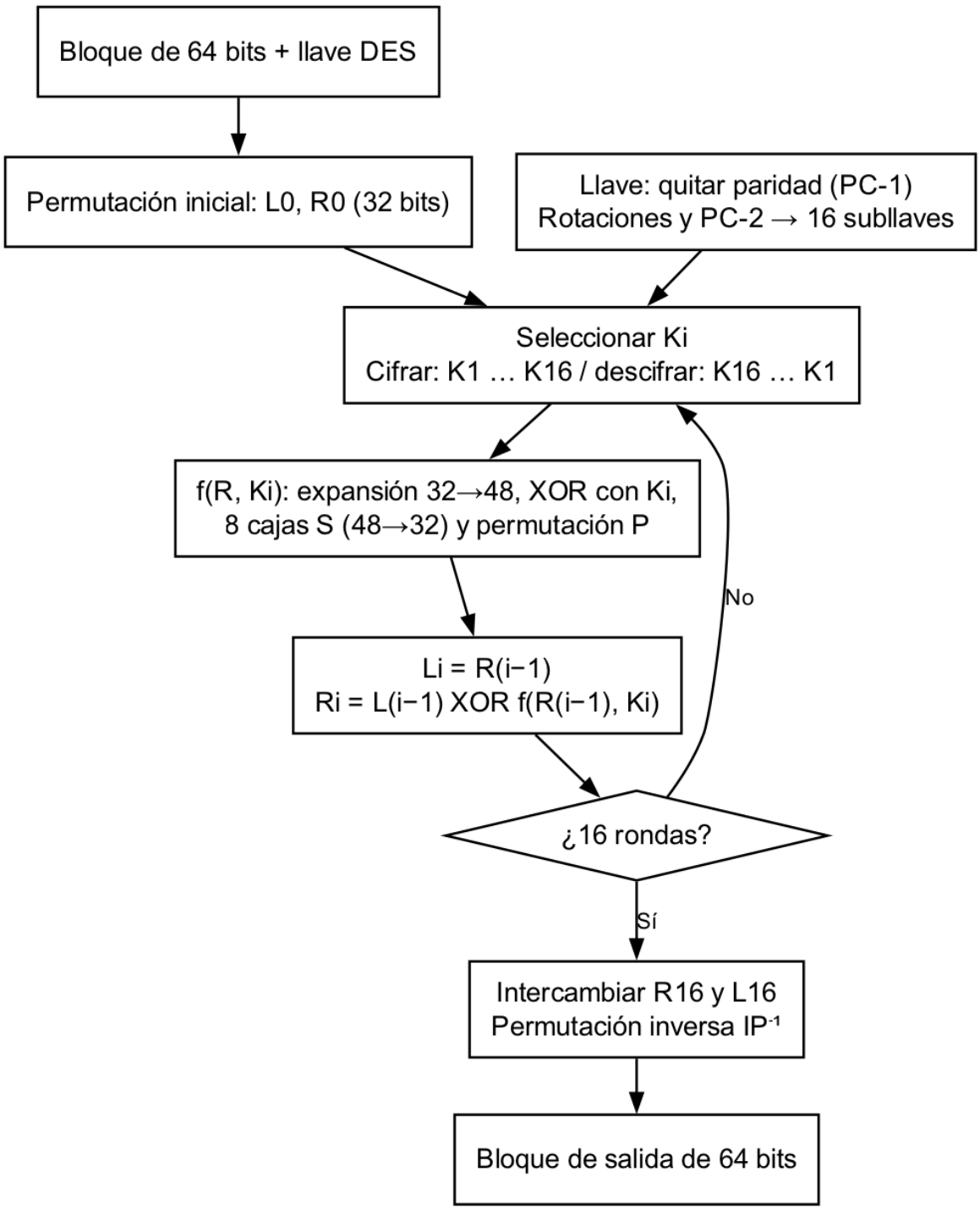


Figura 1. Transformación de un bloque DES. Elaboración propia a partir de FIPS 46-3 [1].

3. Implementación y rutinas

La solución sustituyó `rpc/des_crypt.h` por OpenSSL. El código actual separa el cifrador, la búsqueda secuencial y la búsqueda MPI. Comparte validación, reparto y transformación DES en dos encabezados. Se representa cada llave efectiva con `uint64_t` y se reservan ocho grupos de siete bits para formar los bytes DES, ajustando paridad [4].

Archivo	Función
cifrar.cpp	Archivo .txt → ciphertext DES-ECB; máximo 350 palabras.
bruteforce_seq.cpp	Referencia secuencial; búsqueda incremental.
bruteforce_mpi.cpp	Bloques/cíclico y terminación coordinada.
des_utils.h / app_utils.h	DES, opciones, archivos, reparto y resultados.
benchmarks/	Mediciones, profiling, validación y gráficas.

Rutinas solicitadas

encrypt y decrypt: preparan el calendario y procesan bloques de ocho bytes mediante `DES_ecb_encrypt`, con modo de cifrado o descifrado. Las copias a bloques locales evitan depender de la alineación del buffer [4].

tryKey: copia el ciphertext a un buffer reutilizable, descifra con el candidato y devuelve si aparece la frase. Se usa `std::copy` para copiar y `std::search` para buscar por longitud, incluso después de bytes nulos.

memcpy: copia exactamente n bytes, sin interpretar caracteres. Se utiliza dentro de la transformación de bloques. **strstr:** en el programa base busca una subcadena en texto terminado en NUL; devuelve un puntero o `nullptr`. El programa actual usa búsqueda por longitud porque un descifrado arbitrario puede contener NUL antes de la frase.

Archivo publicado por el equipo

Publicamos en Canvas el archivo `textoequipo10.bin`, cifrado con DES-ECB, con la palabra clave `calabaza` y el rango de búsqueda [0, 500000). Lo generamos con la primera versión del programa, que representaba la llave como un entero de ocho bytes. En la versión actual usamos índices efectivos de 56 bits; la llave 173800 de la primera versión corresponde al índice efectivo 27124.

Repositorio

El código fuente, los scripts de medición y la documentación están disponibles en nuestro repositorio: https://github.com/Giosantos23/Proyecto2Paralela.

3.1. Diagramas de rutinas

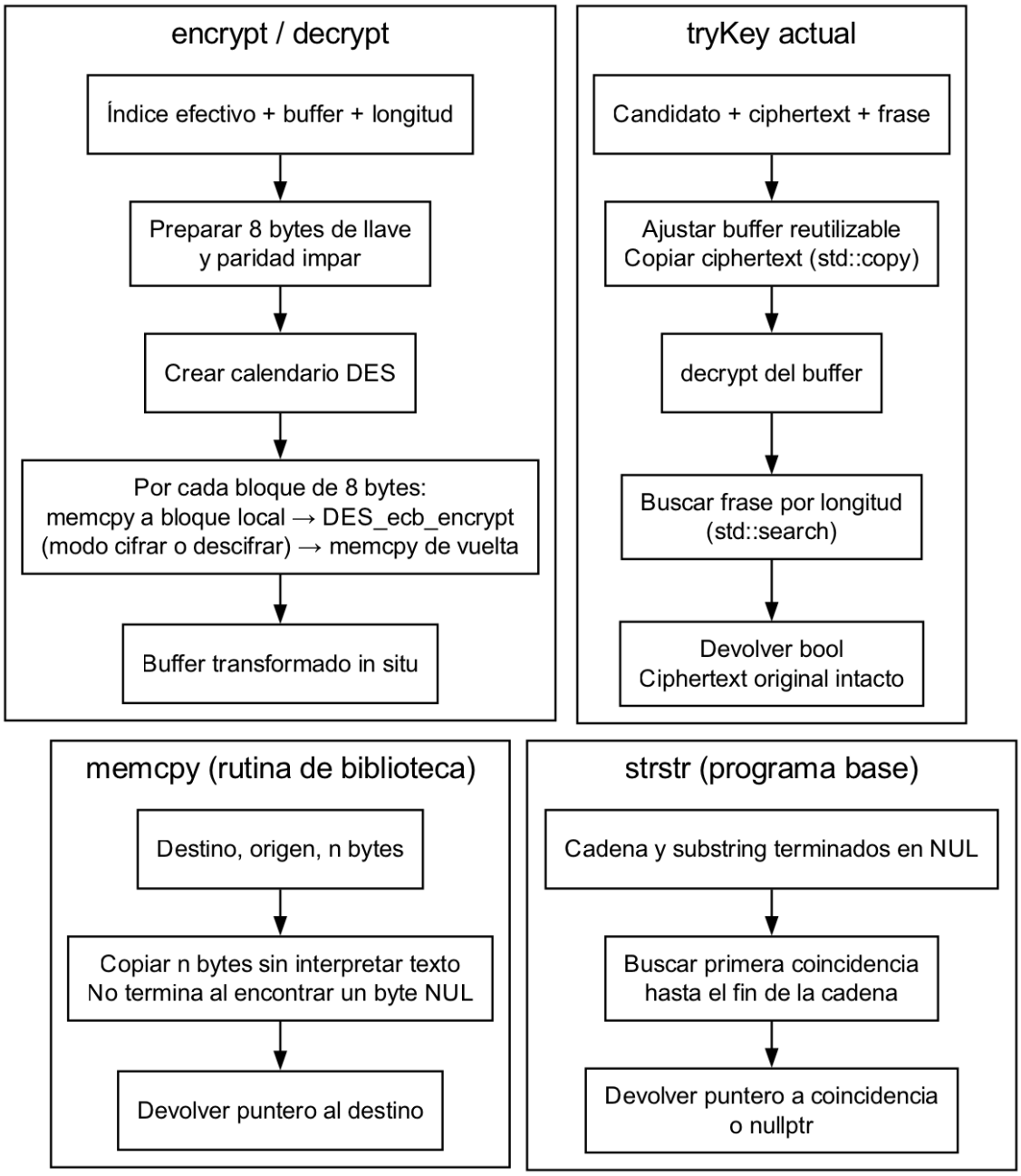


Figura 2. Flujo de transformación, prueba de candidato y rutinas de biblioteca.

4. Flujo del programa MPI

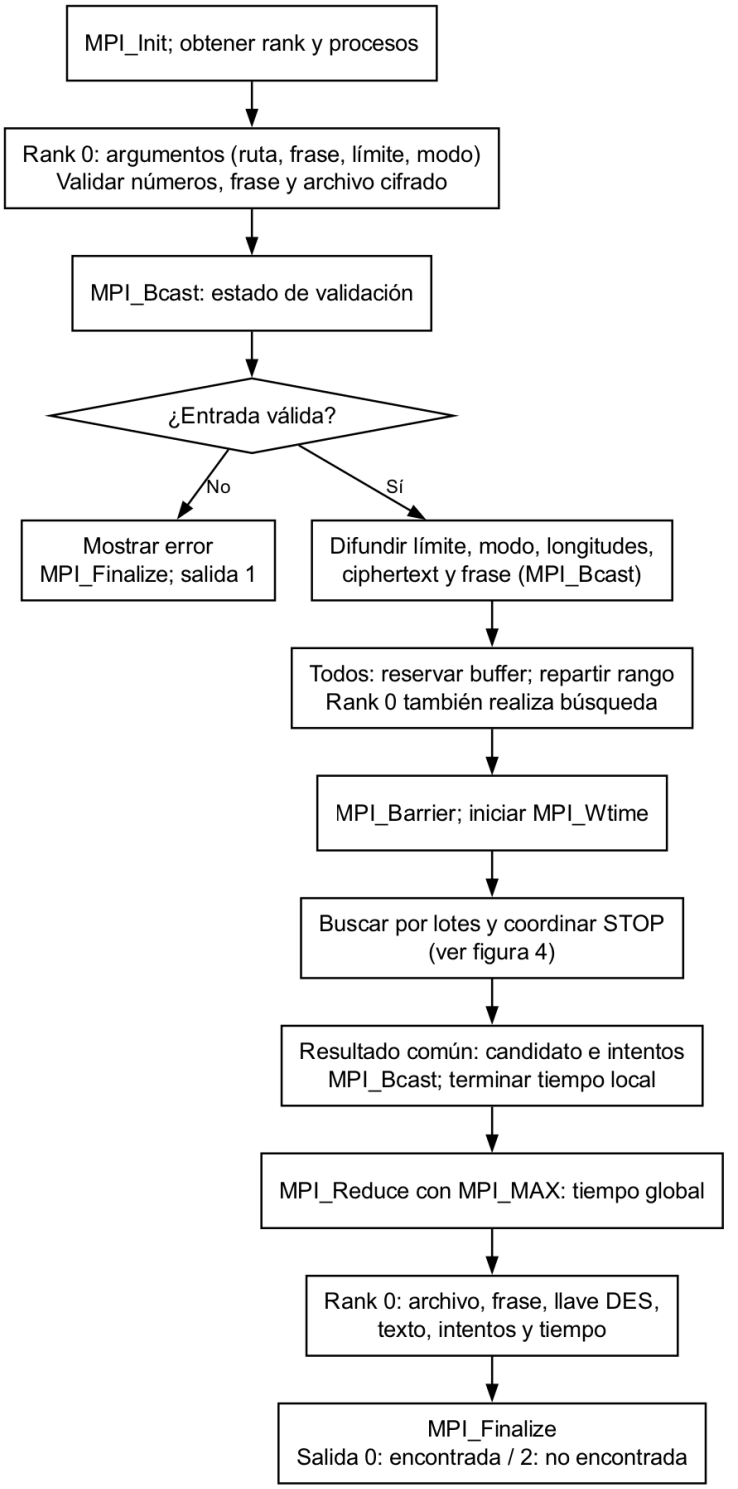


Figura 3. Entrada, programación defensiva, distribución, medición y salida.

4.1. Búsqueda y terminación

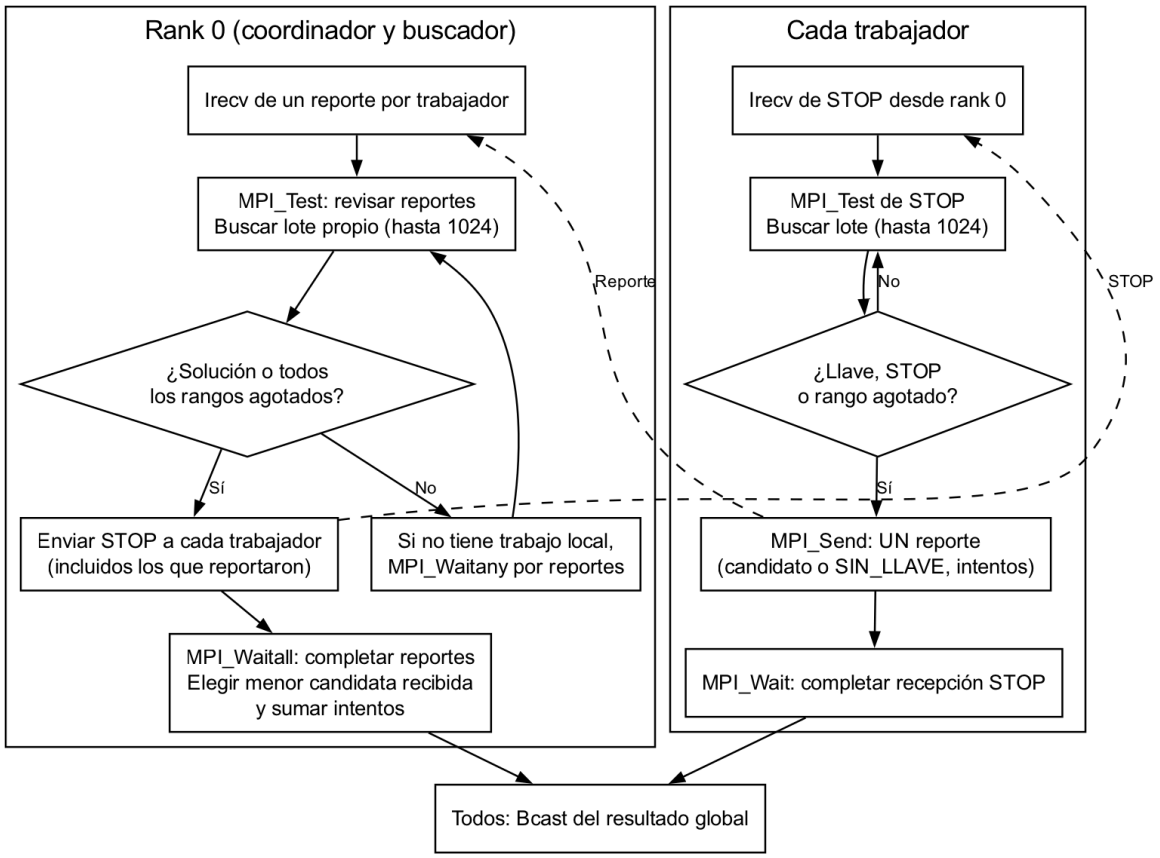


Figura 4. Coordinación del rank 0 y cada trabajador. Flechas discontinuas: mensajes.

Cada trabajador reporta una vez, aunque agote su rango o reciba STOP. El coordinador envía STOP también a quienes ya reportaron y completa todas las recepciones. Si hay hallazgos simultáneos, selecciona la menor candidata reportada; no promete la menor llave de todo el espacio.

4.2. Primitivas y reparto

Primitiva	Uso en el programa
MPI_Irecv	Publica recepción sin esperar su finalización. Rank 0 recibe reportes; trabajadores reciben STOP.
MPI_Send	Envía reportes y STOP. Su retorno permite reutilizar el buffer, sin asegurar que el receptor ya lo haya procesado.
MPI_Wait / MPI_Test	Wait completa una recepción; Test consulta progreso entre lotes. Un request completado queda nulo.
MPI_Waitany / MPI_Waitall	Rank 0 espera reportes cuando no tiene trabajo local y completa todos antes del resultado final.
MPI_Bcast	Difunde validación, entradas y resultado global.
MPI_Barrier / MPI_Wtime / MPI_Reduce	Sincroniza inicio, mide tiempos locales y obtiene el máximo con MPI_MAX.

Las recepciones se publican antes de los envíos correspondientes. La prueba con envíos síncronos comprueba que el protocolo no dependa del buffering. La finalización de operaciones se trata conforme a la semántica MPI [5, 6].

Reparto por bloques

Para límite L y N procesos: $q = \text{floor}(L/N)$, $e = L \bmod N$. El rank r comienza en $r \cdot q + \min(r,e)$ y recibe $q + 1$ llaves si $r < e$; en otro caso, q. Los segmentos cubren [0,L), con diferencias de tamaño de a lo sumo una llave.

Reparto cíclico

El rank r prueba r, $r + N$, $r + 2N$, … mientras el candidato sea menor que L. Se distribuye el recorrido incremental entre procesos. Este cambio busca reducir la dependencia de la solución respecto de los límites de bloques; su mejora se evalúa experimentalmente.

Programación defensiva

Se rechazan argumentos numéricos inválidos, frases vacías, archivos no disponibles, longitudes vacías o incompatibles con DES y tamaños que desbordan los contadores MPI. Los errores de entrada se difunden antes de terminar. Una excepción inesperada durante ejecución llama a `MPI_Abort` para evitar procesos esperando indefinidamente.

5. Método experimental

Condición	Registro
Equipo	Apple M2; 8 núcleos físicos/lógicos; macOS 26.2 arm64.
Compilación	Apple clang 15.0.0; C++17; -O2.
Bibliotecas	OpenMPI 5.0.11; OpenSSL 3.6.5.
Procesos	2, 4, 6 y 8; localhost; sin sobresuscripción ni afinidad fija.
Energía	Modo de bajo consumo activado. Se conservaron registros de energía y carga.
Repeticiones	10 por configuración; 1 calentamiento excluido; orden aleatorio reproducible, semilla 3069.
Tiempo comparado	Búsqueda y terminación. Excluye lectura, difusión inicial, arranque de mpirun e impresión.

El secuencial usa `steady_clock`. MPI sincroniza el inicio, mide con `MPI_Wtime` y reporta el máximo tiempo local. Se calcula el cociente de promedios, no el promedio de cocientes. Se conservan todas las mediciones y se reporta desviación estándar muestral.

Casos controlados

Mantuvimos `prueba.txt`, frase `jacaranda` y L = 4194304 = 2^22. La actividad 9 se adaptó al rango manejable conservando posiciones relativas: fácil L/2 + 1 = 2097153; intermedia L/2 + L/8 = 2621440; difícil ceil(L/7) + ceil(L/13) = 921826. Los nombres describen los ejemplos por bloques con cuatro procesos, no una dificultad absoluta.

Textos externos y profiling

Conservamos los archivos originales y los rangos publicados de los grupos 1 y 7. Verificamos sus hashes y contrastamos los descifrados con OpenSSL. El profiling se ejecutó por separado: muestreo CPU y tres repeticiones de 24 configuraciones MPI. Sus tiempos instrumentados no se usan para recalcular speedups.

Límites del experimento

No se controló la carga de otras aplicaciones ni la asignación a núcleos específicos. Los casos se ejecutaron uno a la vez. Los rangos reducidos no demuestran viabilidad de agotar 2^56; la adaptación de la actividad 9 debe indicarse al evaluar los resultados.

6. Resultados con llaves controladas

Caso	N	Tseq (s)	S bloques	E bloques (%)	S cíclico	E cíclica (%)
facil	2	2.9931	2138.40	106920.1	1.98	99.2
facil	4	2.9931	986.43	24660.7	3.62	90.5
facil	6	2.9931	738.28	12304.6	4.83	80.5
facil	8	2.9931	645.39	8067.4	5.34	66.8
intermedia	2	3.6540	4.87	243.7	1.94	97.1
intermedia	4	3.6540	4.55	113.7	3.61	90.2
intermedia	6	3.6540	3.94	65.6	4.83	80.6
intermedia	8	3.6540	625.96	7824.6	5.77	72.1
dificil	2	1.3225	1.00	49.8	1.99	99.6
dificil	4	1.3225	0.94	23.6	3.77	94.3
dificil	6	1.3225	3.43	57.1	4.81	80.2
dificil	8	1.3225	1.69	21.2	4.93	61.7

Con cuatro procesos, el cíclico produjo speedups de 3,62×, 3,61× y 3,77×. Por bloques produjo 986,43×, 4,55× y 0,94×. La ubicación de la solución explica esta diferencia: en el caso fácil se encuentra al inicio de un bloque, mientras el secuencial debe recorrer más de dos millones de candidatos.

El cíclico resultó más consistente entre estas posiciones. Su eficiencia disminuyó al aumentar procesos: con ocho fue aproximadamente 62–72%. Estos datos no identifican por sí solos una causa de esa disminución.

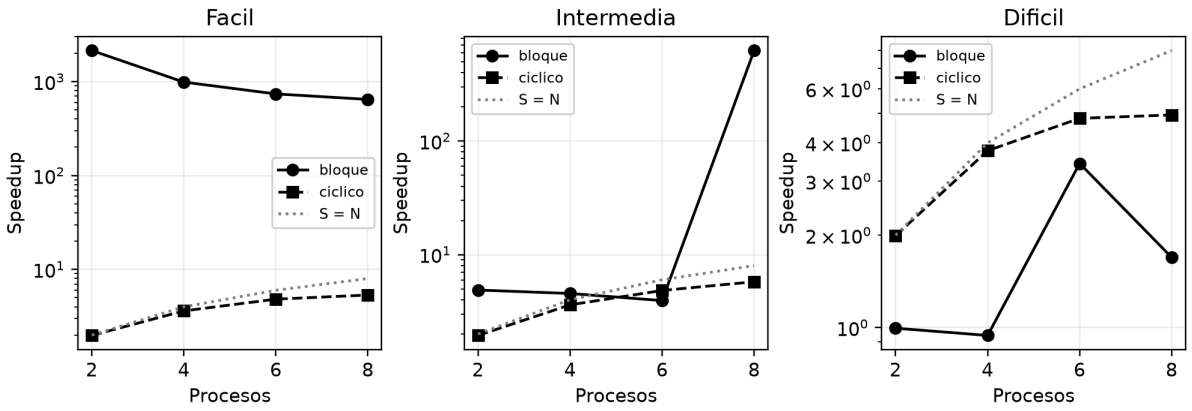


Figura 5. Speedup por posición de llave; escala logarítmica. Fuente: 270 mediciones propias.

7. Descifrado de otros equipos

Grupo	Archivo / bytes	Frase	Rango	Llave efectiva
1	equipo1_mensaje.bin / 816	quetzal sincronizado en MPI	[0, 200000)	173421
7	textoGrupo7.bin / 992	se le olvida como manejar	[0, 4194304)	3412587

Recuperamos ambas llaves por fuerza bruta dentro del rango publicado. Sus bytes DES fueron `da94150101010101` y `d649a10201010101`. Las frases aparecieron en los textos completos y coincidieron con el descifrado independiente. Realizamos 90 mediciones por texto, superando el mínimo de diez pruebas por caso.

Grupo	N	S bloques	E bloques (%)	S cíclico	E cíclica (%)
1	2	2.31	115.3	1.95	97.7
1	4	6.32	158.0	3.52	87.9
1	6	16.93	282.2	4.42	73.7
1	8	4.42	55.2	5.37	67.2
7	2	2.53	126.4	1.95	97.5
7	4	11.49	287.2	3.72	93.1
7	6	4.46	74.3	4.85	80.8
7	8	9.02	112.8	5.77	72.1

La referencia secuencial promedio fue 1,244799 s para el grupo 1 y 29,080400 s para el grupo 7. Con ocho procesos cíclicos, los speedups fueron 5,37× y 5,77×. Por bloques, el grupo 1 alcanzó 16,93× con seis procesos y el grupo 7, 11,49× con cuatro; en esos casos se evaluaron menos llaves que en la referencia.

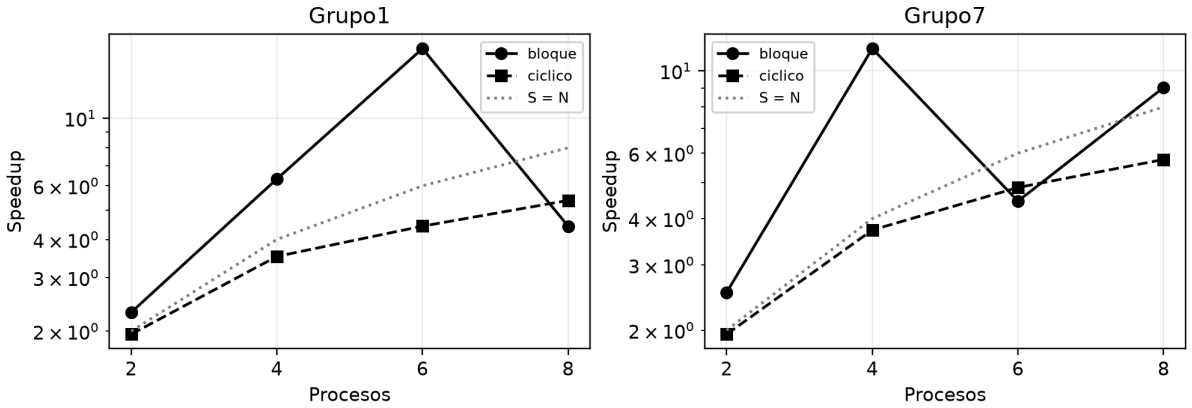


Figura 6. Speedup de los archivos originales de los grupos 1 y 7.

8. Profiling

Muestreamos la búsqueda secuencial durante ocho segundos, con intervalo solicitado de 1 ms, obteniendo 6784 muestras. El ejecutable se compiló con -O2 -g. La tabla usa la función en la cima de la pila para evitar contar simultáneamente padres e hijos.

Función	Muestras	Porcentaje
DES_encrypt1	5794	85.41%
tryKey (código inline)	426	6.28%
DES_set_key_unchecked	327	4.82%
DES_ecb_encrypt	145	2.14%

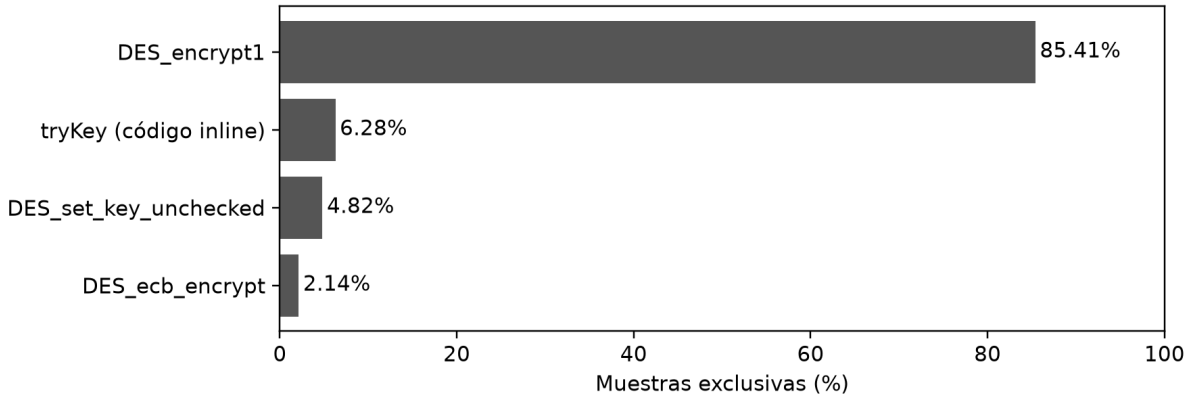


Figura 7. Principales funciones en el muestreo CPU.

El costo principal observado fue la rutina central DES_encrypt1, con 85,41% de las muestras. La preparación del calendario tuvo un peso menor. Parte del código inline queda atribuida a tryKey; no puede asignarse todo ese porcentaje sólo a la búsqueda del substring.

La instrumentación MPI generó 360 registros por rank en 72 ejecuciones. En las configuraciones cíclicas, la búsqueda ocupó al menos 99.11% del tiempo acumulado por proceso, y las llamadas MPI como máximo 0.88%. Se midieron lotes, intentos y duración/ conteo de Irecv, Test, Send, Wait, Waitany, Waitall y Bcast.

El tiempo dentro de MPI incluye progreso y espera; no mide transferencia de red pura. Los perfiles breves por bloques muestran una proporción mayor de coordinación por el pequeño trabajo de búsqueda. No se midieron contadores de memoria ni núcleos asignados, por lo que no se atribuye la pérdida de eficiencia a una única causa.

9. Retos y verificación

Reto	Solución aplicada
Dependencia de rpc/des_crypt.h	Uso de OpenSSL y transformación por bloques locales.
Índices duplicados por paridad	Mapeo uno a uno de los 56 bits efectivos; compatibilidad --legacy.
Bloqueos al encontrar o agotar rango	Una recepción por reporte y STOP, finalización de requests y resultado común.
Búsqueda truncada por NUL	std::search por longitud en lugar de strstr para candidatos arbitrarios.
Speedup dependiente de la llave	Comparación con reparto cíclico y registro de intentos.
Perturbación por instrumentación	Binarios de profiling separados de las mediciones de rendimiento.

Pasaron las pruebas internas de DES y tres suites de integración: ejecutable normal, envíos síncronos y ejecutable MPI instrumentado, con 173 ejecuciones por suite. Se verificaron vector DES conocido, paridad, 56 bits, 65536 índices sin duplicados, compatibilidad histórica, reparto completo y casos sin solución.

La integración incluyó errores de entrada, archivos malformados, más procesos que llaves, hallazgos simultáneos y palabras después de bytes nulos. El ensayo con límite 2ˆ56 usa una solución temprana: valida la cota, sin recorrerla exhaustivamente.

10. Conclusiones y recomendaciones

1. La solución recuperó las llaves de los grupos 1 y 7 y coordinó la terminación MPI en los casos verificados.
2. El reparto por bloques presentó speedups muy sensibles a la ubicación de la llave. Registrar intentos fue necesario para interpretar valores superiores al número de procesos.
3. El reparto cíclico ofreció resultados más consistentes entre las posiciones controladas. En textos externos, con ocho procesos alcanzó $5{,}37\times$ y $5{,}77\times$, con eficiencias de 67,2% y 72,1%.
4. El profiling concentró el costo en DES. Para estos casos cíclicos locales, las llamadas MPI no dominaron el tiempo acumulado por proceso.

Recomendaciones

Usar el reparto cíclico como referencia de consistencia, conservar los datos individuales y comparar con el mismo archivo, frase y límite. Si se optimiza el programa, revisar primero el costo DES por candidato y medir el efecto del tamaño de lote sobre comunicación y trabajo extra.

Para evaluar escalabilidad adicional, controlar carga, energía y afinidad, ampliar textos y llaves y realizar pruebas entre máquinas. Mantener separadas las corridas instrumentadas. Confirmar con el docente la adaptación del rango de la actividad 9.

Referencias

[1] National Institute of Standards and Technology. (1999). Data Encryption Standard (DES), FIPS 46-3. Documento archivado.

https://csrc.nist.gov/files/pubs/fips/46-3/final/docs/fips46-3.pdf

[2] National Institute of Standards and Technology. (2001). Data Encryption Standard. En A Century of Excellence in Measurements, Standards, and Technology (pp. 250–253).

https://nvlpubs.nist.gov/nistpubs/sp958-lide/html/250-253.html

[3] Dworkin, M. (2001). Recommendation for Block Cipher Modes of Operation: Methods and Techniques. NIST SP 800-38A.

https://doi.org/10.6028/NIST.SP.800-38A

[4] OpenSSL Project. (s. f.). DES_random_key y rutinas DES. Documentación OpenSSL 3.0.

https://docs.openssl.org/3.0/man3/DES_random_key/

[5] MPI Forum. (2015). MPI: A Message-Passing Interface Standard, versión 3.1. Semantics of Point-to-Point Communication.

https://www.mpi-forum.org/docs/mpi-3.1/mpi31-report/node58.htm

[6] MPI Forum. (2015). MPI 3.1. Communication Completion.

https://www.mpi-forum.org/docs/mpi-3.1/mpi31-report/node64.htm

Consulta de fuentes: 14 de octubre de 2026. Los datos experimentales se encuentran en los CSV y logs del proyecto. Las publicaciones de los grupos externos se consultaron en Canvas y se documentaron en textos_otros_grupos/README.md.

Anexo 1. Catálogo de funciones

Tipos: KeyType = uint64_t; string = std::string; vector = std::vector. Uso: ciph = ciphertext; keyword = frase; trabajo = buffer temporal; len = bytes; rank = identificador; size = procesos. Los errores recuperables usan std::runtime_error.

preparar_llave (des_utils.h)

Entradas: indice: KeyType; key: DES_cblock&. **Salidas:** key: 8 bytes con paridad impar; void. **Propósito:** Distribuye los 56 bits en ocho grupos de siete bits. Evita índices duplicados por paridad.

llave_legacy_a_efectiva (des_utils.h)

Entradas: antigua: KeyType. **Salidas:** KeyType: índice efectivo. **Propósito:** Interpreta los bytes little-endian anteriores y elimina los bits de paridad.

llave_hex (des_utils.h)

Entradas: indice: KeyType. **Salidas:** std::string: 16 dígitos hexadecimales. **Propósito:** Presenta los bytes reales de la llave DES para comparar implementaciones.

transformar_des (des_utils.h)

Entradas: indice: KeyType; texto: char*; len: size_t; modo: int. **Salidas:** texto modificado in situ; void. **Propósito:** Prepara calendario DES y transforma todos los bloques de ocho bytes, usando bloques locales y memcpy.

encrypt / decrypt (des_utils.h)

Entradas: key: KeyType; texto: char*; len: size_t. **Salidas:** texto cifrado / descifrado; void. **Propósito:** Invocan transformar_des con DES_ENCRYPT o DES_DECRYPT. La longitud debe ser múltiplo de ocho.

tryKey (des_utils.h)

Entradas: key: KeyType; ciph: const vector<char>&; keyword: const string&; trabajo: vector<char>&. **Salidas:** bool: aparece la frase; trabajo: texto descifrado. **Propósito:** Reutiliza buffer, copia ciphertext, descifra y busca por longitud. No modifica ciph.

leer_numero (app_utils.h)

Entradas: texto: const string&; nombre: const char*; minimo, maximo: KeyType. **Salidas:** KeyType validado; excepción runtime_error. **Propósito:** Acepta decimal completo sin signo ni sufijos, dentro del rango inclusivo solicitado.

Anexo 1. Catálogo de funciones (continuación 1)

leer_opciones **(app_utils.h)**

Entradas: argc: int; argv: char**; paralelo: bool. **Salidas:** OpcionesBusqueda; excepción ante error. **Propósito:** Valida cantidad de argumentos, frase, límite exclusivo y modo bloque/ciclico.

leer_archivo **(app_utils.h)**

Entradas: ruta: const string&; cifrado: bool. **Salidas:** vector<char> con bytes; excepción ante error. **Propósito:** Comprueba apertura, tamaño positivo, cota segura y múltiplo de ocho para ciphertext.

repartir **(app_utils.h)**

Entradas: limite: KeyType; rank, size: int; ciclico: bool. **Salidas:** Rango {siguiente, fin, paso}: KeyType. **Propósito:** Asigna un segmento equilibrado o una progresión por rank. Cubre el rango sin huecos ni duplicados.

imprimir_entrada **(app_utils.h)**

Entradas: op: const OpcionesBusqueda&; len: size_t. **Salidas:** stdout; void. **Propósito:** Muestra archivo, frase, rango y longitud.

imprimir_resultado **(app_utils.h)**

Entradas: llave: KeyType; ciph: const vector<char>&; intentos: KeyType. **Salidas:** stdout; void. **Propósito:** Muestra candidato, hexadecimal, descifrado y total de intentos, o ausencia de solución.

buscar_lote **(bruteforce_mpi.cpp)**

Entradas: rango: Rango&; ciph: const vector<char>&; keyword: const string&; trabajo: vector<char>&; reporte: Reporte&. **Salidas:** Actualiza cursor, candidato e intentos; void. **Propósito:** Prueba hasta 1024 llaves y regresa para permitir atender comunicación.

buscar_en_rango **(bruteforce_mpi.cpp)**

Entradas: rank, size: int; rango: Rango; ciph: const vector<char>&; keyword: const string&; trabajo: vector<char>&. **Salidas:** Reporte: array<KeyType,2> común. **Propósito:** Coordina reportes y STOP, completa requests y difunde candidato e intentos globales.

Anexo 1. Catálogo de funciones (continuación 2)

main: cifrar (cifrar.cpp)

Entradas: argc: int; argv: char**. **Salidas:** int: 0 éxito / 1 error; archivo .bin. **Propósito:** Valida archivo y máximo de 350 palabras, añade espacios, cifra y verifica escritura.

main: secuencial (bruteforce_seq.cpp)

Entradas: argc: int; argv: char**. **Salidas:** int: 0 encontrada / 2 ausente / 1 error. **Propósito:** Recorre [0, límite), cuenta intentos y mide búsqueda con steady_clock.

main: MPI (bruteforce_mpi.cpp)

Entradas: argc: int; argv: char**. **Salidas:** int: código común; resultado en rank 0. **Propósito:** Inicializa MPI, valida/difunde datos, reparte, mide con Wtime y MAX, muestra y finaliza.

OpcionesBusqueda / Rango / Reporte (app_utils.h y bruteforce_mpi.cpp)

Entradas: ruta y keyword: string; limite: KeyType; ciclico: bool; cursores: KeyType. **Salidas:** Datos de opciones, reparto y reporte. **Propósito:** Reporte contiene candidato o SIN_LLAVE e intentos. SIN_LLAVE = 2^56 queda fuera de las llaves válidas.

perfil_iniciar / perfil_lote (benchmarks/profile_mpi.cpp)

Entradas: Sin argumentos / segundos: double. **Salidas:** Estado activo / tiempo acumulado; void. **Propósito:** Activa medición PMPI y acumula tiempo por lote sólo en el binario instrumentado.

perfil_finalizar (benchmarks/profile_mpi.cpp)

Entradas: rank, size: int; intentos: unsigned long long; total: double; ciclico: bool. **Salidas:** CSV por rank; void. **Propósito:** Desactiva medición, reúne datos fuera de la ventana medida y escribe tiempos/conteos.

Wrappers MPI_* (benchmarks/profile_mpi.cpp)

Entradas: Argumentos de las primitivas MPI. **Salidas:** Código int de PMPI; tiempos y conteos locales. **Propósito:** Interceptan Irecv, Test, Send, Wait, Waitany, Waitall y Bcast sin cambiar el protocolo.

Anexo 2. Bitácora: casos controlados

Todos los tiempos son segundos internos de búsqueda. R1–R10 identifican las diez repeticiones medidas; no incluyen calentamientos. Texto: prueba.txt; frase: jacaranda; límite: 4194304. Datos completos, intentos y comandos: resultados/rendimiento-10-repeticiones-2026-10-14/mediciones.csv.

Caso fácil: llave 2097153

Configuración	R1	R2	R3	R4	R5	R6	R7	R8	R9	R10
secuencial N = 1	2.94665	2.93563	3.18728	2.92300	2.99098	2.96557	3.01142	3.02295	2.96089	2.98683
bloque N = 2	0.00156	0.00002	0.00153	0.00152	0.00153	0.00157	0.00167	0.00147	0.00161	0.00152
ciclico N = 2	1.48757	1.48793	1.56092	1.52453	1.52191	1.51853	1.49954	1.51564	1.48775	1.48718
bloque N = 4	0.00300	0.00301	0.00300	0.00468	0.00300	0.00303	0.00300	0.00299	0.00154	0.00309
ciclico N = 4	0.96370	0.83062	0.78855	0.79333	0.81432	0.78479	0.80306	0.91090	0.79375	0.78494
bloque N = 6	0.00302	0.01088	0.00300	0.00300	0.00300	0.00566	0.00299	0.00300	0.00300	0.00300
ciclico N = 6	0.57810	0.59921	0.62586	0.67320	0.65833	0.61655	0.63757	0.59766	0.60593	0.60789
bloque N = 8	0.01003	0.00253	0.00449	0.00300	0.00383	0.00240	0.00300	0.00933	0.00273	0.00504
ciclico N = 8	0.51759	0.53199	0.67551	0.59188	0.57123	0.51267	0.60636	0.51111	0.53345	0.55204

Caso intermedio: llave 2621440

Configuración	R1	R2	R3	R4	R5	R6	R7	R8	R9	R10
secuencial N = 1	3.66386	3.67193	3.66215	3.64046	3.63912	3.66044	3.67864	3.64257	3.63381	3.64705
bloque N = 2	0.74445	0.74409	0.76273	0.74581	0.75087	0.74825	0.74988	0.74418	0.74549	0.76012
ciclico N = 2	1.89676	1.93348	1.85756	1.86492	1.87285	1.87474	1.87336	1.87975	1.88918	1.87425
bloque N = 4	0.84936	0.78544	0.81679	0.77672	0.81227	0.79351	0.81469	0.77811	0.82932	0.77896
ciclico N = 4	1.01854	1.00546	0.97855	0.97876	0.99774	1.04358	1.02203	0.99551	1.08288	1.00111
bloque N = 6	0.95439	0.88772	0.96783	0.91941	0.90820	0.92638	0.89892	0.97287	0.92667	0.92136
ciclico N = 6	0.77425	0.72749	0.72736	0.77823	0.72955	0.74871	0.75724	0.75301	0.80605	0.75699
bloque N = 8	0.00261	0.01211	0.00281	0.00423	0.01444	0.00966	0.00254	0.00486	0.00271	0.00240
ciclico N = 8	0.62999	0.65155	0.63792	0.64999	0.60859	0.66620	0.59190	0.67154	0.59987	0.62968

Anexo 2. Bitácora: continuación

Caso difícil: llave 921826

Configuración	R1	R2	R3	R4	R5	R6	R7	R8	R9	R10
secuencial N = 1	1.27271	1.27475	1.27577	1.28570	1.30340	1.31255	1.29187	1.29255	1.63969	1.27605
bloque N = 2	1.30965	1.30697	1.33010	1.31018	1.30304	1.37632	1.33169	1.32882	1.34117	1.34787
ciclico N = 2	0.67192	0.65142	0.65940	0.67429	0.65536	0.65947	0.67160	0.65769	0.65596	0.67971
bloque N = 4	1.36170	1.42426	1.42877	1.45295	1.43214	1.38226	1.37889	1.36538	1.39055	1.41236
ciclico N = 4	0.37473	0.34366	0.34233	0.35212	0.34600	0.34214	0.35303	0.34954	0.35191	0.34987
bloque N = 6	0.37992	0.37761	0.40172	0.37825	0.38625	0.38159	0.38830	0.39674	0.39535	0.37200
ciclico N = 6	0.24718	0.26819	0.25566	0.28146	0.27300	0.29101	0.29365	0.30861	0.25553	0.27538
bloque N = 8	0.76870	0.76710	0.82659	0.74878	0.85150	0.74551	0.78776	0.71522	0.75162	0.84818
ciclico N = 8	0.23008	0.23521	0.29042	0.24141	0.27856	0.25454	0.25012	0.25432	0.42360	0.22244

Anexo 2. Bitácora: textos externos

Archivos originales. Grupos 1 y 7: diez repeticiones por configuración, 180 mediciones y 18 calentamientos excluidos. Intentos y comandos: resultados/otros-grupos-2026-10-14/mediciones.csv.

Grupo 1: límite 200000; llave 173421

Configuración	R1	R2	R3	R4	R5	R6	R7	R8	R9	R10
secuencial N = 1	1.23684	1.26636	1.22224	1.25604	1.29494	1.24988	1.22782	1.22061	1.22902	1.24424
bloque N = 2	0.53219	0.53582	0.53404	0.53697	0.53328	0.55256	0.53540	0.57002	0.53511	0.53399
ciclico N = 2	0.63013	0.63235	0.63214	0.62820	0.63625	0.67629	0.64294	0.62883	0.63221	0.63227
bloque N = 4	0.19082	0.18824	0.18431	0.18372	0.19425	0.18339	0.18497	0.19755	0.19057	0.27236
ciclico N = 4	0.35418	0.34604	0.38305	0.36604	0.34258	0.34042	0.34106	0.34685	0.36801	0.35027
bloque N = 6	0.06371	0.07380	0.06818	0.08224	0.07301	0.07698	0.06714	0.08950	0.06445	0.07612
ciclico N = 6	0.25547	0.27693	0.28286	0.28342	0.27323	0.27738	0.36448	0.27203	0.26057	0.26746
bloque N = 8	0.33128	0.25461	0.24967	0.24294	0.25812	0.24319	0.26784	0.42046	0.27035	0.27843
ciclico N = 8	0.25463	0.22131	0.23212	0.24259	0.23712	0.24119	0.20791	0.22721	0.22756	0.22489

Grupo 7: límite 4194304; llave 3412587

Configuración	R1	R2	R3	R4	R5	R6	R7	R8	R9	R10
secuencial N = 1	29.02940	28.93220	29.16160	28.94730	29.29110	28.97480	28.99770	29.02520	29.10840	29.33630
bloque N = 2	11.58810	11.43180	11.43430	11.49110	11.43570	11.63100	11.48270	11.44550	11.67330	11.43540
ciclico N = 2	14.86310	14.84150	14.84050	14.82850	14.81390	15.58150	14.82780	14.84350	14.81960	14.83260
bloque N = 4	2.46632	2.43943	2.87680	2.51935	2.44896	2.53313	2.50927	2.49199	2.51742	2.51031
ciclico N = 4	7.79735	7.77556	7.73323	7.83371	7.80570	7.79125	8.05289	7.75548	7.74964	7.78615
bloque N = 6	6.53781	6.67589	6.45033	6.44236	6.54875	6.46228	6.40027	6.51920	6.62286	6.60278
ciclico N = 6	5.92284	5.91689	6.21418	6.05816	6.04240	6.02743	5.91356	5.96282	5.98342	5.91254
bloque N = 8	3.18623	3.17510	3.12408	3.14785	3.20241	3.00256	3.23146	3.10477	3.14478	3.91096
ciclico N = 8	4.86014	4.88232	5.00021	5.11524	5.54257	5.04331	4.77835	4.97628	5.25375	4.94005

Apéndice. Evidencia y reproducción

Archivos de respaldo

Carpeta / archivo	Contenido
resultados/rendimiento-10-repeticiones-2026-10-14/	270 registros, resúmenes, entorno y logs.
resultados/otros-grupos-2026-10-14/	180 registros finales, descifrados, fuentes y hashes.
resultados/profiling-2026-10-14/	Muestreo CPU, 360 registros por rank, resúmenes y regresiones.
textos_otros_grupos/	Ciphertexts originales y transcripción de las publicaciones.

Compilación y validación

```
make
make test
make test-sync
make profile
```

Los comandos completos de cada experimento están en los CSV. Los scripts y sus parámetros se describen en benchmarks/README.md y benchmarks/PROFILING.md. Se conservan logs y datos originales para revisar promedios, desviaciones, speedup y eficiencia.

Integridad de textos externos

Grupo 1
71f4a8281f89b0529be845b1ea3aa1fcc7334577fa3a8d16c12d66f3a75f0e39

Grupo 7
edda33babaef3fa7274981fefa3f8cf1df53f5f08ec46d8d7c504f218db933d0

Los hashes se verificaron al finalizar. El descifrado del grupo 1 conserva tres bytes de relleno 0x03 y el del grupo 7 tres bytes nulos en las copias .raw; las copias legibles .txt retiran ese relleno.

Repositorio del proyecto

https://github.com/Giosantos23/Proyecto2Paralela